TKS V8.0: A TYPED FINITE-FUNCTION KERNEL FOR COGNITIVE-ARCHITECTURE SEMANTICS, WITH EXACT LIFT THEORY AND DENOTATIONAL NORMAL FORMS

LITTLE HENDERSON

ABSTRACT. We develop, in full, a deliberately small executable reference kernel for the typed-composition layer of the Tootra Knowledge System (TKS) and its secular derivative, the Cognitive Architecture System (CAS). The kernel supplies five capabilities, each specified and proved here: (1) named finite objects with explicitly ordered carriers, whose identity discipline eliminates hash-order nondeterminism; (2) total finite-function arrows with declared source and target, and partial, type-checked categorical composition satisfying the category laws; (3) a hard type separation between one-carrier endomaps, which admit monoid powers and the exact idempotence theorem $f^{|X|!} \circ f^{|X|!} = f^{|X|!}$, and heterogeneous semigroupoid paths, which have composites but deliberately no powers; (4) a deterministic standard-basis lift $A_f e_x = e_{f(x)}$ into finite-dimensional Hilbert spaces, for which we prove the exact operator-norm formula $\|A_f\|^2 = \max_y |f^{-1}(y)|$ via a fiber-block Gram decomposition, the trace, rank, and isometry corollaries, functoriality, and a permanent noncontractivity counterexample; and (5) stable denotational normal forms by shortlex minimization over a completed finite closure, with a soundness proof for representative-first pruning and explicit resource semantics. We document the executable evidence exactly—bounded exhaustive suites covering 19,683 associativity triples, 289 endomaps, 499 typed maps, and 1,678 lift-functor composition pairs—and the pinned Lean 4.32.1 formal seed with its verified and open scope. The kernel maps no canonical Noetic or CAS construct onto arrows and makes no empirical claim; the complete proof-obligation ledger, integration boundaries, and reproduction procedure are included.

CONTENTS

1. Introduction 2
1.1. Motivation 2
1.2. Contributions and their evidential status 2
2. The typed finite-function category 3
2.1. Objects 3
2.2. Arrows and composition 3
2.3. Endomaps, powers, and paths 4
3. The deterministic standard-basis lift 5

Date: August 7, 2026.

Research paper v0.2, 2026-08-07 (supersedes white paper v0.1). Artifact status: DRAFT / GATE 1 PROTOTYPE / RING 4 / NOT LICENSED / NOT FROZEN. Source basis: `TKS_V8_CORE_FORMAL_SPEC_AND_PROOF_OBLIGATIONS_v0.1.md` (SHA-256 `F740BA05AB7ADB2DCDF55E271016D127E45073B015497DE8982D44A49F0EA2C4`); prototype manifest TKS-V8-TYPED-CORE-PROTOTYPE v0.1.0; theorem-ledger records THM-LIFT-NORM-FIBER-FORMULA, THM-LIFT-OCCUPATION-COUNTEREXAMPLE, THM-FINITE-IDEMPOTENT-POWER; implementation `src/tks_v8_core/` (`category.py` 9,850 B, `deterministic_lift.py` 5,483 B, `normal_forms.py` 9,889 B, hashes in the manifest); verifier `scripts/verify_tks_v8_gate1.py`. Drafting this paper promotes no claim; independent specialist review of every result remains open.

3.1. Definition and elementary structure 5
3.2. The exact norm theorem 5
3.3. Functoriality 6
4. Stable denotational normal forms 6
4.1. Typed words and shortlex 6
4.2. The closure algorithm and its soundness 6
5. Executable evidence 7
5.1. The formal seed 7
6. Proof and integration ledger 8
7. Integration boundaries and downstream role 8
7.1. What is excluded 8
7.2. Downstream boundary objects 8
8. Reproduction 9
9. Related work 9
10. Conclusion 9
References 9

1. Introduction

1.1. **Motivation.** The TKS/CAS formal program (v7.4 through the v7.5–v7.7 extensions) defines a rich ontology—ten Noetics, four Domains, seven Drives, four Registers, and a disciplined operation notation—but its production notation compiler enforces value *kinds* (state, drive, relation, program) rather than categorical *source and target objects*. The Project Coherence mathematical red-team audit identified three concrete failure modes that a kind system cannot exclude:

(1) *Untyped composition.* Nothing prevents composing operations whose intermediate objects disagree; the result is a well-formed expression with no denotation.
(2) *Invalid power inference.* A sequence of heterogeneous, time-varying generators is not an element of one finite semigroup. Treating a generator sequence as "one operator iterated n times" licenses false fixed-point and idempotence conclusions—for instance, importing Theorem 2.9 below into a setting where no single endomap exists.
(3) *Nondeterministic identity.* If carrier order is left to hash order, basis indices, denotations, and canonical representatives depend on interpreter state (`PYTHONHASHSEED`), destroying replayability of every downstream digest.

TKS v8.0 is the smallest kernel that turns each failure into a type error. It is an independent reference model: it does not modify, replace, or certify the active CAS notation compiler, and it deliberately refuses to encode the TKS/CAS ontology before that mapping is separately reviewed (Section 7).

1.2. **Contributions and their evidential status.** Sections 2–4 develop the mathematics with complete proofs. Every claim is tagged with its registered status: the category laws and lift functoriality carry partial formal verification in a pinned Lean seed; the norm theorem carries an informal proof and exhaustive structural tests, with universal formalization open; the shortlex results carry an informal correctness argument; the factorial-power theorem is a standard result whose formalization is open. Section 5 states the executable evidence exactly. Section 6 reproduces the full proof-obligation ledger. Nothing in this paper is peer-reviewed,

and no empirical, cognitive-dynamics, channel-contractivity, or CAS-efficacy claim follows from any result.

2. The typed finite-function category

2.1. Objects.

Definition 2.1 (Object)**.** An *object* is a pair $X = (\mathrm{name}_X, \mathrm{carrier}_X)$ where $\mathrm{carrier}_X$ is a finite, explicitly ordered sequence of distinct hashable elements. Object identity is the pair: two objects are equal iff their names and their carrier sequences (with order) are equal. We write $|X|$ for the carrier length and index carrier positions $0, \ldots, |X| - 1$, calling these the *standard indices*.

Two design decisions deserve justification.

Name-inclusive identity. Carrier-only identity would silently coerce semantically distinct objects of equal carrier—for example, two different registers that both carry $\{0, 1\}$. Composition legality (Definition 2.3) must track intent, not just arity, so the declared name participates in identity.

Ordered-input discipline. Carrier elements, arrow mappings, path arrows, word labels, catalog objects, catalog generators, and lift coordinate vectors all reject sets, mappings, and one-shot iterables at construction. Their order changes basis indices (Section 3), typed denotations, and shortlex rank (Section 4); silently accepting hash-order iteration would make results depend on interpreter state. Callers must provide a replayable ordered sequence. This rule is enforced in the constructors of `category.py` and tested by negative fixtures.

2.2. Arrows and composition.

Definition 2.2 (Arrow)**.** An *arrow judgment* $f : X \to Y$ consists of declared source X, declared target Y, and a total map assigning to every standard index of X one standard index of Y, represented as the tuple $(f(0), \ldots, f(|X| - 1))$. Arrows are *extensionally equal* when source, target, and mapping tuple agree; presentation names (Section 2.3) never affect equality.

Definition 2.3 (Identity and composition)**.** For each object X, id_X is the arrow with $\mathrm{id}_X(i) = i$. For $f : X \to Y$ and $g : Y' \to Z$, the composite $g \circ f$ is defined *iff* $Y = Y'$ as objects (name and carrier), in which case

$$(g \circ f)(i) = g(f(i)) : X \to Z.$$

No cardinality-only or carrier-only coercion is performed; a source/target mismatch raises a typed error.

Theorem 2.4 (Category laws; status: partial formal verification present)**.** *Under Definitions 2.1–2.3, identities are neutral and composition is associative wherever defined; the objects and arrows form a category* **FinArr**.

Proof. Neutrality: $(f \circ \mathrm{id}_X)(i) = f(\mathrm{id}_X(i)) = f(i)$ and $(\mathrm{id}_Y \circ f)(i) = \mathrm{id}_Y(f(i)) = f(i)$ for every standard index i; sources and targets agree by construction. Associativity: if $f : X \to Y$, $g : Y \to Z$, $h : Z \to W$, then both $(h \circ g) \circ f$ and $h \circ (g \circ f)$ are defined, have source X and target W, and satisfy

$$((h \circ g) \circ f)(i) = h(g(f(i))) = (h \circ (g \circ f))(i).$$

Extensional equality of arrows is exactly equality of these data. □

Remark 2.5 (Verification scope). The executable suite checks Theorem 2.4 exhaustively on a bounded family: all $19{,}683 = (3^3)^3$ composable triples of endo-arrows on a fixed three-point carrier, plus typed families across distinct carriers (499 typed maps with source and target sizes through four). A pinned Lean 4.32.1 module (`TypedCategory`) kernel-checks the identity and associativity lemmas for the Lean model of these definitions, under the single standard axiom `propext`, with project axioms and `sorry` rejected by the verifier. The equivalence between the Python implementation and the Lean model is itself an open obligation (PO-01/PO-07 in Section 6); neither bounded testing nor the seed constitutes verification of the running code.

2.3. **Endomaps, powers, and paths.** The kernel's central discipline is a type-level separation of two constructions that informal operator notation conflates.

Definition 2.6 (Endomap)**.** Endomap(f) is constructible only when $f : X \to X$ with source and target the *same* object. It carries the monoid powers f^n for $n \geq 0$, with $f^0 = \mathrm{id}_X$ and $f^{m+n} = f^m \circ f^n$.

Definition 2.7 (Semigroupoid path)**.** A *semigroupoid path* is a nonempty sequence

$$X_0 \xrightarrow{f_1} X_1 \xrightarrow{f_2} \cdots \xrightarrow{f_n} X_n$$

whose adjacent objects match (Definition 2.3). It has the typed composite $f_n \circ \cdots \circ f_1 : X_0 \to X_n$ and *intentionally no power operation*: the constructor exposes no $(\,\cdot\,)^n$, and the runtime types are distinct, so a path cannot be passed where an endomap is required.

Remark 2.8 (Why the separation is load-bearing). Suppose a workflow applies generators $g_1, g_2, \ldots$ in temporal order across changing state spaces. Nothing guarantees the g_t are equal, or even endo. Any conclusion of the form "after enough steps the process is idempotent" requires a *single* finite endomap (Theorem 2.9); for a path, the very expression p^2 is ill-typed unless $X_0 = X_n$, and even then p need not equal any single generator. The kernel makes the invalid inference unwritable rather than merely discouraged.

Theorem 2.9 (Safe finite power; ledger THM-FINITE-IDEMPOTENT-POWER; status: standard result, formalization open)**.** *For every finite endomap $f : X \to X$, the power $f^{|X|!}$ is idempotent:*

$$f^{|X|!} \circ f^{|X|!} = f^{|X|!}.$$

Proof. Write $N = |X|$ and consider the functional graph of f on the standard indices: each weakly connected component consists of trees hanging on a unique cycle. For any point x, the tail length (distance from x to its cycle) is at most $N - 1$, so after $N! \geq N - 1$ (for $N \geq 2$; the cases $N \in \{0, 1\}$ are immediate) iterations, $f^{N!}(x)$ lies on a cycle. Let $\ell \leq N$ be that cycle's length. Since $\ell \mid N!$, a further $N!$ iterations traverse the cycle a whole number of times:

$$f^{2N!}(x) = f^{N!}(x).$$

As x was arbitrary, $f^{2N!} = f^{N!}$, which is the claim. □

Remark 2.10. The suite verifies Theorem 2.9 exhaustively for all $289 = \sum_{k \leq 4}$ (per-carrier) endomaps on carriers through size four. Lean formalization is open (PO-04).

Construction 2.11 (Bounded presentation names)**.** Exponentiation uses binary powering. Presentation names for intermediate powers are bounded: large exponents are rendered by bit length plus a short SHA-256 label rather than full decimal conversion, so name construction is $O(\log n)$ in the exponent and cannot hit the host language's decimal-digit conversion limit.

Regression tests cover $f^{2^{30}}$ and $f^{10^{5000}}$. Presentation names never enter extensional equality (Definition 2.2).

3. The deterministic standard-basis lift

3.1. Definition and elementary structure.

Definition 3.1 (Lift)**.** For $f : X \to Y$, let H_X and H_Y be complex Hilbert spaces with orthonormal bases $(e_x)_{x\in X}$ and $(e_y)_{y\in Y}$ indexed by the ordered carriers. The *deterministic standard-basis lift* is the linear map

$$A_f : H_X \to H_Y, \qquad A_f e_x = e_{f(x)}.$$

In matrix form, $(A_f)_{yx} = 1$ if $f(x) = y$ and 0 otherwise: a column-stochastic 0/1 matrix with exactly one 1 per column.

Three operators derived from A_f are kept distinct in the implementation, because informal usage tends to conflate them:

(1) $A_f^* A_f$, the *input-collision Gram matrix*: $(A_f^* A_f)_{xx'} = 1$ iff $f(x) = f(x')$;
(2) $A_f A_f^*$, diagonal with entries the fiber sizes $|f^{-1}(y)|$;
(3) the *codomain support projector*, the diagonal 0/1 matrix marking nonempty fibers—not equal to $A_f A_f^*$ unless every fiber has size ≤ 1.

3.2. The exact norm theorem.

Theorem 3.2 (Exact operator norm; ledger THM-LIFT-NORM-FIBER-FORMULA; status: informal proof present, universal formalization open)**.** *For nonempty X, with the empty-domain value defined as zero,*

$$\|A_f\|^2 = \max_{y\in Y} \big|f^{-1}(y)\big|, \qquad \|A_f\| = \sqrt{\max_{y\in Y} |f^{-1}(y)|}.$$

Proof. $\|A_f\|^2$ is the largest eigenvalue of the positive semidefinite $A_f^* A_f$. By the computation above, $(A_f^* A_f)_{xx'} = 1$ iff x, x' lie in the same fiber of f. Reorder the input basis grouping fibers together (a permutation similarity, which preserves spectrum): the Gram matrix becomes a direct sum $\bigoplus_{y\in \mathrm{im} f} J_{m_y}$ of all-ones blocks, $m_y = |f^{-1}(y)|$. The all-ones matrix J_m has rank one with $J_m^2 = mJ_m$; its eigenvalues are m (eigenvector the all-ones vector) and 0 with multiplicity $m - 1$. Hence the spectrum of $A_f^* A_f$ is $\{m_y\}_{y\in \mathrm{im} f} \cup \{0\}$ (with multiplicities), and its maximum is $\max_y m_y$. □

Corollary 3.3 (Implemented consequences)**.** $\mathrm{tr}(A_f^* A_f) = |X|$*;* $\mathrm{rank}(A_f) = |\mathrm{im}(f)|$*; and A_f is an isometry iff f is injective.*

Proof. The trace is the sum of fiber sizes, which is $|X|$ by totality. The columns of A_f span $\{e_y \mid y \in \mathrm{im} f\}$, giving the rank. Isometry means $A_f^* A_f = I$, i.e. every fiber has size exactly one on the diagonal and no off-diagonal collisions—precisely injectivity. □

Example 3.4 (Permanent noncontractivity fixture; ledger THM-LIFT-OCCUPATION-COUNTEREXAMPLE)**.** Let f be the constant map on a two-point carrier. Then

$$A_f = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad A_f^2 = A_f, \qquad \|A_f\| = \sqrt{2}, \qquad \mathrm{tr}(A_f^* A_f) = 2, \qquad \mathrm{rank} A_f = 1.$$

A_f is idempotent yet not an orthogonal projector (it is not self-adjoint), not a partial isometry, and has norm exceeding one. The fixture permanently blocks two tempting general claims:

"deterministic lifts are contractive" and "idempotent lifts are projectors." The Lean seed kernel-checks this exact two-state expansion and its noncontractivity.

3.3. **Functoriality.**

Theorem 3.5 (Lift functor laws; status: partial formal verification, finite-sum matrix bridge open)**.** *$A_{\mathrm{id}_X} = I_{H_X}$ and, whenever $g \circ f$ is defined, $A_{g\circ f} = A_g A_f$. Thus $A_{(-)}$ is a functor from* **FinArr** *to finite-dimensional Hilbert spaces and linear maps.*

Proof. On basis vectors, $A_{\mathrm{id}_X} e_x = e_x$ and $A_g A_f e_x = A_g e_{f(x)} = e_{g(f(x))} = A_{g\circ f} e_x$; linear extension determines the maps. Functor laws for composition of three or more arrows follow from Theorem 2.4. □

Remark 3.6 (Exact verification scope)*.* Functoriality is checked by *exact matrix equality* (integer entries, no floating point) for every valid typed-map pair on carriers through size three: 1,678 composition pairs including empty carriers. The Lean seed kernel-checks the identity lift entrywise and on basis vectors, typed composition on basis vectors, and relational-product support composition. Open in Lean: finite-sum numerical matrix multiplication, the generic bridge from Example 3.4 to matrix/Gram/trace/rank definitions, and arbitrary-vector linearity (PO-05, PO-11).

Claim Boundary 3.7*.* The lift is a bookkeeping functor for finite deterministic maps. No stochastic kernel, quantum channel, contractivity, or dynamical claim follows; Example 3.4 exists precisely to prevent the contractivity reading.

4. Stable denotational normal forms

4.1. **Typed words and shortlex.**

Definition 4.1 (Generator catalog and typed words)**.** A *catalog* is a finite, explicitly ordered sequence $G = (g_0, \dots, g_{k-1})$ of typed generators (arrows of **FinArr**) over a finite object set, together with the declared order, which assigns each generator its *rank*. A *typed word* is a start object X_0 together with a sequence of generator labels whose arrows compose in sequence from X_0. The empty word at X denotes id_X; a nonempty word denotes the composite of its arrows. We write $[\![w]\!]$ for the denotation.

Definition 4.2 (Shortlex)**.** For typed words u, v from the same start object, $u <_{\mathrm{slex}} v$ iff $|u| < |v|$, or $|u| = |v|$ and the rank sequence of u is lexicographically smaller. For a reachable denotation a,

$$\mathrm{NF}_G(a) = \min_{<_{\mathrm{slex}}} \{\, w \text{ well-typed} \mid [\![w]\!] = a \,\}.$$

4.2. **The closure algorithm and its soundness.**

Algorithm 4.3 (Representative-first closure)**.** Explore typed words in shortlex order (breadth-first by length, lexicographic by rank within a length). Maintain a table from extensional denotations to their first-discovered word. When a word's denotation is already in the table, record the duplicate and *do not expand it*; otherwise store it and enqueue its one-generator extensions. Terminate when the frontier is empty; return the table.

Lemma 4.4 (Pruning soundness)**.** *Algorithm 4.3 computes NF_G on every reachable denotation.*

Proof. Two invariants hold by induction on the exploration order. (i) Words are dequeued in shortlex order: extensions preserve relative shortlex order of their prefixes, since appending

a fixed label adds one to both lengths and appends the same rank. (ii) If prefixes $u <_{\text{slex}} u'$ satisfy $[\![u]\!] = [\![u']\!]$, then for any suffix s making $u's$ well-typed, us is well-typed (denotations determine the object at the split point), $[\![us]\!] = [\![u's]\!]$, and $us <_{\text{slex}} u's$. Hence pruning u' removes no denotation and no shortlex-least representative: any word extending a pruned prefix is dominated by the corresponding extension of the retained representative. The first-discovered word for each denotation is therefore its shortlex minimum. □

Proposition 4.5 (Finiteness)**.** *The closure terminates: for finite objects X, Y there are at most $|Y|^{|X|}$ total maps $X \to Y$, and the catalog's object set is finite, so the set of reachable denotations is finite and each is expanded at most once.*

Construction 4.6 (Resource semantics)**.** The implementation returns only after closure completion. It deduplicates queued candidates by typed denotation and separately caps accepted denotations and the pending frontier; exceeding either bound raises `ClosureLimitExceeded`. A partial table is *never* returned as canonical—the fail-closed alternative to silently truncated "canonical" forms.

Claim Boundary 4.7. Two boundaries are explicit and permanent. (i) NF_G is denotational minimization over a completed finite closure—*not* a claim that any inherited TKS rewrite system terminates or is confluent; no rewrite system is defined here. (ii) Canonical forms are relative to the declared, versioned generator order: changing the order may change representatives without changing denotations. Cross-catalog comparison of normal forms is meaningless without a catalog identity.

5. Executable evidence

The validation snapshot of 2026-08-04 records PASS on 20 tests with the following bounded exhaustive coverage:

Bounded exhaustive family	Cases
Three-point associativity triples	19,683
Endomaps on carriers through size four	289
Typed maps, source and target sizes through four	499
Lift-functor composition pairs through size three (exact matrices)	1,678

The test suite comprises four modules bound by SHA-256 in the manifest: category and power regressions; lift structure and functor-law regressions; normal-form correctness and resource regressions; and a manifest-boundary regression that asserts the integration boundary of Section 7 (active compiler unmodified, no production runtime dependency, canonical TKS v8 status OPEN, semantic transfer OPEN / NOT LICENSED).

5.1. **The formal seed.** A pinned, isolated Lean 4.32.1 toolchain (commit `f054605a...`) builds through Lake under `scripts/verify_psr8_lean_gate1.py`, which hardcodes the sole allowed standard axiom (`propext`), rejects project axiom declarations and `sorry`, checks source hashes, and audits exact theorem dependency reports. Verified scope: typed total-function identities and associativity with endomap-only powers; the exact two-state constant-lift expansion and noncontractivity (Example 3.4); deterministic-lift identity entrywise and on basis vectors; typed basis composition; relational-product support composition; and the four-atom finite-context lemmas used by the companion Noetic Topos paper. Open scope: the universal norm theorem (Theorem 3.2), the generic matrix/Gram/trace/rank bridge, finite-sum matrix multiplication, arbitrary-vector linearity, Theorem 2.9, shortlex leastness (Lemma 4.4), and all presheaf/sheaf formalization.

Claim Boundary 5.1. Passing the bounded executable suite establishes only the behavior covered by this finite executable model. It is not proof-assistant verification of the Python implementation, not independent specialist review, and closes no open obligation.

6. Proof and integration ledger

ID	Obligation	Status
PO-TKS8-CORE-01	Identity and associativity for finite-function composition	partial formal verification present (pointwise proof; bounded exhaustive tests; exact Lean lemmas)
PO-TKS8-CORE-02	Reject every source/target mismatch	executable validation (constructor and negative tests)
PO-TKS8-CORE-03	Keep one-carrier powers distinct from heterogeneous paths	executable validation (separate runtime types; negative tests)
PO-TKS8-CORE-04	$f^{\lvert X\rvert !}$ idempotent for finite endomaps	standard result; formalization open
PO-TKS8-CORE-05	Exact deterministic-lift norm formula	informal proof present; universal formalization open (Lean checks only the two-state expansion)
PO-TKS8-CORE-06	Soundness and leastness of shortlex catalog	informal correctness argument; formalization open
PO-TKS8-CORE-07	Independent theorem-prover formalization	partial, present (pinned seed with axiom audit)
PO-TKS8-CORE-08	Reconcile kernel object/arrow types with corrected v7.4 theorem records	open; not licensed (no semantic mapping admitted)
PO-TKS8-CORE-09	Integrate with the active notation compiler	open (outside this lane; requires compatibility design and review)
PO-TKS8-CORE-10	Independent specialist review	open (no external review receipt)
PO-TKS8-CORE-11	Lift preserves identities and composition	partial formal verification; finite-sum matrix bridge open

7. Integration boundaries and downstream role

7.1. **What is excluded.** Excluded and still OPEN: parsing or evaluating TKS v7.4/v8 surface syntax; mapping the ten canonical Noetics or any CAS construct to kernel arrows; RPM monad data or laws; a rewrite system for inherited TKS equations; Noetic Topos 2.0 coverage and modalities; stochastic kernels, quantum channels, or empirical dynamics; and any transfer from the Ten Advances manuscript. All such research transfers remain Ring 4 / NOT LICENSED.

7.2. **Downstream boundary objects.** Three companion candidates consume the kernel strictly through its types:

(1) *CAS Core 8* bridges legacy fractal digit sequences through an explicit caller-supplied catalog of typed arrows; composition uses semigroupoid paths, and missing digits or mismatches fail closed. The bridge proves type-correct composition only, not the meaning of a legacy digit.

(2) *CAS PCBA 8* may adapt only a certified total, finite, deterministic, stateless, non-stochastic component transition to a kernel arrow; chains become paths; feedback powers require a certified endomap. The adapter certificate binds the exact kernel bytes and carries no automatic CAS semantic theorem.
(3) *Noetic Topos 2.0* shares the type discipline (deterministic maps admitted; Markov kernels, channels, and enriched sheaves deliberately not identified with functions) and its future reconciliation with kernel objects is exactly PO-08.

8. Reproduction

From the repository root with `src` on `PYTHONPATH`:

```
python -m unittest discover -s tests/tks_v8_core -v
python scripts/verify_tks_v8_gate1.py
python scripts/verify_psr8_lean_gate1.py
```

The Gate 1 verifier checks artifact hashes against the manifest and reruns the executable suite; the Lean verifier reproduces the formal seed under the pinned toolchain. The manifest does not hash itself; a later reviewed commit or acceptance receipt must bind the manifest SHA-256, and until then this is an uncommitted Gate 1 candidate, not a freeze receipt.

9. Related work

The categorical laws, the finite-semigroup idempotent-power fact, and the spectral analysis of 0/1 column-selection matrices are standard mathematics [1, 2, 3]; the kernel's contribution is an *engineering discipline*—name-inclusive object identity, ordered-input rejection of hash-order nondeterminism, the endomap/path type separation, fail-closed closure semantics, and hash-bound artifacts—rather than new theorems. Property-preserving executable models with bounded exhaustive testing follow the small-scope tradition [4]; the pinned single-axiom Lean seed follows standard kernel-checking practice [5]. No novelty search has been commissioned; the ledger records all mathematical content as standard results or definitions.

10. Conclusion

TKS v8.0 contributes a disciplined foundation rather than a sweeping theory: typed composition that cannot silently mistype (Theorem 2.4), powers that exist only where a monoid exists (Definitions 2.6–2.7, Theorem 2.9), a lift whose exact norm is proved rather than assumed (Theorem 3.2) and whose one permanent counterexample blocks the two standard over-readings (Example 3.4), and canonical forms that are deterministic because their inputs are ordered and their closure is completed or refused (Lemma 4.4, Construction 4.6). The kernel's refusal to encode the TKS/CAS ontology before review is deliberate: the mapping of canon onto substrate is itself a gated research obligation (PO-08), and preserving that boundary—with the same precision as the proofs—is part of the result.

References

[1] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.
[2] J.-É. Pin, "Finite semigroups and recognizable languages: an introduction," in *Semigroups, Formal Languages and Groups*, Kluwer, 1995.
[3] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2013.
[4] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, rev. ed., MIT Press, 2012.
[5] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," *CADE 28*, 2021.
[6] Project Coherence, *Mathematical Red-Team Report*, v0.1.

[7] Project Coherence, *Theorem Ledger*, v0.1.